

Tekninen yhteistyökeskustelu

Nuorten jalkapallon talenttiarviointi- ja kehitysseuranta-SaaS

Laadittu 24.6.2026 · Tero Koskela · Luottamuksellinen — jaettavaksi vain keskustelukumppanille

1. Tilannekuva

TalentMaster on suomalainen jalkapallon talenttiarviointi- ja kehitysseuranta-SaaS, jota on tähän asti rakennettu **yhden tekijän + AI-avustajan** moodissa. Tuote on **aktiivisessa pilotissa** (8+2 seuraa; dataa tuotu viidelle: SJK, GrIFK, Sibbo-Vargarna, Pallo-lirot, KPV — yhteensä ~530 pelaajaa). Tavoite tässä keskustelussa: arvioida (a) voiko toinen kehittäjä tulla turvallisesti mukaan, ja (b) saada ulkopuolinen näkemys skaalautuvuudesta, tietoturvasta ja teknologiavalinnoista.

Ydinviesti: Luuranko — multi-tenant-datamalli, palvelinpuolen valtuutus ja domain-logiikka — on oikea ja kantaa. Skaalautuvan SaaS:n este ei ole arkkitehtuurin perusvalinnat, vaan loppuun viemätön insinöörikuri (frontendin modulaarisuus, deploy-automaatio, GDPR-prosessit). Suuri osa pohjatyöstä on jo tehty.

2. Tekninen arkkitehtuuri

Kerros	Teknologia
Frontend	Vanilla JS (IIFE), monisivuinen HTML, ei frameworkia. Roolikohtaiset näkymät.
Tietokanta	Firebase Firestore (eur3 multi-region). Multi-tenant: <code>seurat/{seuraid}/...</code>
Autentikointi	Firebase Auth + Custom Claims (rooli, seuraid) + Google Sign-In; pelaajat PIN→Anonymous.
Backend	Cloud Functions (Node 22, europe-west1), 18 funktiota. API-avaimet Secret Managerissa.
Tallennus / sähköposti	Firebase Storage (kuvat); SendGrid/Nodemailer Cloud Functionissa.
Hosting	GitHub Pages + Fastly CDN. Versiointi <code>version.json</code> + <code>cache-bust</code> .

Datamalli & domain

- Neljä pääkokoelmaa** + multi-tenant-eristys seuratasolla. **Pikakenttä-periaate:** raakadata talletetaan, indeksit lasketaan; dashboard lukee valmiit pikakentät pelaajadokumentista — ei alikokoelmakyselyitä renderissä (suorituskyky + kustannus).
- Kolmiportainen roolihierarkia:** platform-admin → seuran johto (valmennuspäällikkö/sihteeri/UTJ) → operatiiviset (valmentajat, fysiikka, fysio, testivastaava) → pelaaja/vanhempi.
- Domain-logiikka** (kanonisissa moduuleissa + testattu): 5D-arviointimalli, kehon valmiusindeksi, tekniikkaindeksit, biologinen ikä (Mirwald/PHV). Tämä on tuotteen tieteellinen ydin ja kilpailuetu.

3. Mitä on rakennettu

- Roolikohtaiset sovellukset: seurahallinta, valmennuspäällikön dashboard, valmentajan työtila, kenttätestaustyökalu (offline-ensin), pelaajan + vanhemman mobiiliapp, massatuonti (Excel + Palloliiton PDF-parseri).
- Mittaus- ja analyysiputki: testituloksista automaattiset pikakentät + joukkue-KPI:t + signaalit.
- Kalenteri (juuri valmistunut): yksilähteinen tapahtumamalli, läsnäolomerkintä kolmelle roolille, toistuvat tapahtumat jaetulla logiikkakirjastolla.

- Suostumus- ja rekisteröintiputki (huoltajakutsu → suostumus → tilien luonti) Cloud Functioneilla.

4. Laadun tila — mitä insinöörikuria on jo maksettu

Aiempi bugihistoria oli pääosin *hiljaisia epäonnistumisia* (puuttuva indeksi, kenttänimi-mismatch, kuollut listener). Suuri osa näistä virheluokista on nyt suljettu rakenteellisesti:

- **Automaattitestit:** Vitest 296 yksikkötestiä (KPI-/indeksilaskenta, normit, kalenterilogiikka) + 97 Security-Rules-testiä (emulaattori). CI ajaa nämä jokaisessa muutoksessa.
- **Index-as-code:** firestore.indexes.json repossa → uusi kysely ei pääse tuotantoon ilman indeksiiä.
- **Havainnoitavuus:** Sentry (EU-region, vain virheet, PII-skrubaus alaikäisdataalle).
- **Salaisuudet:** API-avaimet Google Cloud Secret Managerissa, ei selaimessa eikä plaintextina.
- **Runtime:** Cloud Functions Node 22; funktio-törmäykset purettu; aikaleima-kenttien tyyppivartijat Rulesissa.

Käytännön merkitys: **perusta on nyt turvallinen toiselle kehittäjälle** — testit + CI + Rules-testit estävät hiljaisen rikkomisen, jota manuaalinen tarkistus ei huomannut.

5. Tekninen velka — rehellinen kuva

Velka	Vaikutus	Tila / suunnitelma
Frontend-monoliitit (~6 000 riviä/rooli; markup+CSS+JS yhdessä)	Suurin este kahden kehittäjän rinnakkaistyyölle (merge-konfliktit, ei uudelleenkäyttöä)	Mitigaatio aloitettu: jaettu logiikka eriytetty lib-moduuleiksi. Vite + strangler-malli suunniteltu, ei aloitettu (B1).
Rules deployataan käsin Consolesta (CI saa 403)	Ei automaattista review-porttia tuotanto-Rulesille	Emulaattoritestit (97) ajetaan CI:ssä; deploy manuaalinen. Ratkaistava: service-accountin oikeus → CI-deploy.
Versiointi tiedostonimessä + jäänteitä vanhoista versioista	Sekaannus, copy-paste-riski	Repo-siivous osin tehty; loppuun vietävä.
GDPR: retention / oikeus tulla unohdetuksi / DPIA / DPA:t	Alaikäisten EU-data = suurin riski JA pakko ennen laajaa myyntiä	Suostumushallinta on; retention/RTBF/audit/DPIA puuttuu. Odottaa tietosuojavastaavaa (DPO).
Harjoitepankki-konventiot (4 limittyvää rakennetta)	Refaktoroinnin riski ilman karakterisointitestejä	Eristetty oma sprintiksi; karakterisointitestit ensin.

Pienemmät: per-käyttäjä seen-state localStoragessa (ei cross-device); Cloud Functions 1st-gen → v2 -migraatio; SDK-version pinnaus.

6. Voiko toinen kehittäjä tulla mukaan?

Kyllä — ja pohjatyö sen mahdollistamiseksi on suurelta osin tehty. Realistiset ehdot:

- **Turvaverkko on:** testit + CI + Rules-testit + index-as-code sulkevat juuri ne hiljaiset virheluokat, joihin yksin tekevä sortuu. Uusi kehittäjä saa palautteen ennen tuotantoa.
- **Onboarding-dokumentaatio on:** kanoninen CLAUDE.md (tekninen briefing + invariantit) + arkkitehtuuridokumentit kuvaavat datamallin, roolit ja sudenkuopat.
- **Ennen täyttä rinnakkaistyyötä kannattaa:** (1) viedä frontendin modulaarisuus seuraavalle tasolle (strangler-malli, jaetut moduulit — aloitettu), (2) automatisoida Rules-deploy (poistaa manuaalisen pullonkaulan + deploy-luottamusriskin), (3) sopia selkeä työnjako rooli/feature-pohjalta monoliittien merge-konfliktien välttämiseksi.

Suositeltu ensimmäinen yhteinen työ: frontendin moduulijaon strangler-aloitus + Rules-deploy-automaatio. Nämä poistavat kahden kehittäjän pahimmat kitkat ennen kuin featureita rakennetaan rinnakkain.

7. Kriittiset kysymykset ammattilaiselle

Kysymykset joihin haluan ulkopuolisen näkemyksen:

- **Frontend-strategia:** kannattaako Vite-migraatio (strangler), vai riittääkö monoliittien kevyt jako moduuleihin? ROI ja aikataulu pilotin rinnalla?
- **Skaalautuvuus:** Firestore-kustannus- ja suorituskykymalli kun seuroja/pelaajia on kymmen-satakertaisesti? Denormalisointi-/aggregaattistrategia?
- **GDPR / alaikäisdata:** DPIA, retention ja oikeus tulla unohdetuksi — prioriteetti ja toteutusjärjestys? Milloin pakko ennen myynnin laajennusta?
- **Teknologiavalinnat:** kantaako Firebase + Custom Claims skaalassa, vai tuleeeko kattoon? Mitä kannattaa vaihtaa nyt vs. myöhemmin?
- **Tiimi & prosessi:** miten rakentaa 1+1 kehittäjän turvallinen työnkulku (PR-review, CI-portit, staging/preview-tenant, Rules-deploy)?

8. Tiekartta — lyhyesti

Vaihe	Fokus	Lopputulos
Tekninen velka (P0)	Testit, indeksit, Rules-CI, törmäykset — suurelta osin tehty	Toinen kehittäjä voi osallistua turvallisesti
P1	Frontend-moduulit + havainnoitavuus + tenant-self-service + GDPR-perusta	Skaalautuva alusta, monta seuraa ilman käsityötä
P2	Suorituskyky + datamallin versiointi + tuotteistus (white-label, API, AI-insightit)	Kaupallinen SaaS

9. Mitä otan tapaamiseen mukaan

- Tämä briefing (arkkitehtuuri + tehty työ + tekninen velka + kysymykset).
- Lukuoikeus GitHub-repoon + kanoninen CLAUDE.md (invariantit/onboarding) ja skaalautuvuus-/tekninen-velka-dokumentti.
- Live-demo pilottiseuran datalla (esim. SJK) — todellinen tuote, ei mockup.
- Tämän hetken luvut: pilotti 5 seuraa / ~530 pelaajaa; 296 yksikkötestiä + 97 Rules-testiä.